

OPTA-PROG-IR-TOAERIS

PT-BR

Programação OPTA + Aeris IR

1. Informações Gerais

- Versão / Revisão: Revisão 1
- Data de Desenvolvimento: 29/08/2024
- Autor: Daniel Arcos
- Linguagem: C++
- Plataforma / Hardware: Finder OPTA

2. Objetivo da Programação

O objetivo desta programação é estabelecer comunicação via Modbus com o OPTA e o Aeris IR para controle de ar condicionado

3. Requisitos e Dependências

- Instalar o Arduino IDE em sua última versão disponível
- Instalar os drivers de Hardware do OPTA no Arduino IDE
- Instalar biblioteca do OPTA
- Instalar bibliotecas utilizadas na programação

4. Passo a Passo da Implementação

1. Faça a conexão entre o OPTA e o Aeris IR via Modbus (lembre-se de utilizar um resistor de terminação)

5. Código completo

Todo o arquivo da programação está disponível para download nos arquivos.

Visão Geral

Definições e Includes:

```
#include <ArduinoModbus.h>
#include <ArduinoRS485.h>

constexpr auto baudrate { 38400 };
constexpr auto btime { 1.0f / baudrate };
constexpr auto predl { btime * 9.6f * 3.5f * 1e6 };
constexpr auto postdl { btime * 9.6f * 3.5f * 1e6 };
```

- Inclui as bibliotecas necessárias para comunicação Modbus/RS485
- Define a taxa de comunicação como 38400 baud
- Calcula os tempos de pré e pós-delay para controle RS485

Função setup (Configuração Inicial):

```
void setup() {
    Serial.begin(115200);
    delay(2000); // Espera inicialização

    // Configura delays para controle RS485
    RS485.setDelays(predl, postdl);

    // Inicializa comunicação Modbus RTU
```

```

if (!ModbusRTUClient.begin(baudrate, SERIAL_8N2)) {
    Serial.println("Erro Modbus");
    while (1); // Para em caso de erro
}

Serial.println("Sistema pronto. Envie comandos no formato: <registro> <valor>");
}

```

Função loop (Loop Principal):

```

void loop() {
    // Verifica se há comandos via Serial
    if (Serial.available() > 0) {
        String input = Serial.readStringUntil('\n');
        processCommand(input);
    }

    // Exemplo de leitura de dados (comentado)
    /*
    float p = readdata(0x21, 0X09BA);
    float v = readdata(0x21, 0X09C4);
    float i = readdata(0x21, 0X09D4);

    Serial.println(String(p, 1) + "W " + String(v, 1) + "V " + String(i, 3) + "A ");
    delay(3000);
    */
}

```

Função processCommand (Processamento de Comandos):

```

void processCommand(String input) {
    input.trim();
    if (input.length() == 0) return;

    // Parse do comando: <registro> <valor>

```

```

int spaceIndex = input.indexOf(' ');
if (spaceIndex == -1) {
    Serial.println("Formato inválido. Use: <registro> <valor>");
    return;
}

// Extrai registro e valor
String regStr = input.substring(0, spaceIndex);
String valueStr = input.substring(spaceIndex + 1);

// Converte para inteiros
int reg = regStr.toInt();
int value = valueStr.toInt();

// Envia comando Modbus
if (writeData(0x1, reg, value)) {
    Serial.print("Comando enviado com sucesso para o registrador ");
    Serial.println(reg);
} else {
    Serial.print("Erro ao enviar comando para o registrador ");
    Serial.println(reg);
}
}

```

Função writeData (Escrita Modbus):

```

bool writeData(int addr, int reg, int value) {
    // Escreve no registrador de holding
    if (!ModbusRTUClient.holdingRegisterWrite(addr, reg, value)) {
        Serial.println("Erro ao enviar comando. Código de erro: " +
            String(ModbusRTUClient.lastError()));
        return false;
    }
    return true;
}

```

Função readdata (Leitura - não implementada):

```
// Esta função não está implementada no código fornecido
// Seria necessária para leitura de dados do dispositivo
float readdata(int addr, int reg) {
    // Implementação exemplo:
    // return ModbusRTUClient.holdingRegisterRead(addr, reg);
    return 0.0;
}
```

6. Funcionamento do Sistema

Este código implementa um mestre Modbus RTU com as seguintes características:

Protocolo Modbus RTU:

- **Endereço do Dispositivo:** 0x01 (padrão no código)
- **Baud Rate:** 38400
- **Configuração Serial:** 8 bits de dados, sem paridade, 2 stop bits (SERIAL_8N2)
- **Tipo de Registro:** Holding Registers

Fluxo de Operação:

1. **Inicialização:** Configura comunicação RS485 com delays calculados
2. **Aguardando Comandos:** Fica aguardando entradas do Monitor Serial
3. **Processamento:** Interpreta comandos no formato "<registro> <valor>"
4. **Comunicação:** Envia comandos Modbus para o dispositivo escravo
5. **Feedback:** Informa sucesso ou falha na operação

Exemplo de Uso:

No Monitor Serial, digite:

10 1

O valor acima, ligará o ar condicionado

No Monitor Serial, digite:

15 0

O valor acima, desligará o ar condicionado

EN

OPTA + Aeris IR Programming

1. General Information

- Version / Revision: Revision 1
- Development Date: 29/08/2024
- Author: Daniel Arcos
- Language: C++
- Platform / Hardware: Finder OPTA

2. Programming Objective

The objective of this programming is to establish communication via Modbus with the OPTA and Aeris IR for air conditioning control

3. Requirements and Dependencies

- Install the latest version of the Arduino IDE
- Install the OPTA hardware drivers in the Arduino IDE
- Install the OPTA library
- Install libraries used in programming

4. Implementation Step by Step

1. Make the connection between the OPTA and the Aeris IR via Modbus (remember to use a termination resistor)

5. Full code

The entire C++ programming file is available for download in the archives.

Overview

Definitions and Includes:

```
#include <ArduinoModbus.h>
#include <ArduinoRS485.h>

constexpr auto baudrate { 38400 };
constexpr auto btime { 1.0f / baudrate };
constexpr auto predl { btime * 9.6f * 3.5f * 1e6 };
constexpr auto postdl { btime * 9.6f * 3.5f * 1e6 };
```

- Includes the necessary libraries for Modbus/RS485 communication
- Sets the baud rate to 38400 baud
- Calculates pre- and post-delay times for RS485 control

Setup function (Initial Configuration):

```

void setup() {
  Serial.begin(115200);
  delay(2000); // Espera inicialização

  // Configura delays para controle RS485
  RS485.setDelays(predl, postdl);

  // Inicializa comunicação Modbus RTU
  if (!ModbusRTUClient.begin(baudrate, SERIAL_8N2)) {
    Serial.println("Erro Modbus");
    while (1); // Para em caso de erro
  }

  Serial.println("Sistema pronto. Envie comandos no formato: <registro> <valor>");
}

```

Loop function (Main Loop):

```

void loop() {
  // Verifica se há comandos via Serial
  if (Serial.available() > 0) {
    String input = Serial.readStringUntil('\n');
    processCommand(input);
  }

  // Exemplo de leitura de dados (comentado)
  /*
  float p = readdata(0x21, 0X09BA);
  float v = readdata(0x21, 0X09C4);
  float i = readdata(0x21, 0X09D4);

  Serial.println(String(p, 1) + "W " + String(v, 1) + "V " + String(i, 3) + "A ");
  delay(3000);
  */
}

```


processCommand Function (Command Processing):

```
void processCommand(String input) {
    input.trim();
    if (input.length() == 0) return;

    // Parse do comando: <registro> <valor>
    int spaceIndex = input.indexOf(' ');
    if (spaceIndex == -1) {
        Serial.println("Formato inválido. Use: <registro> <valor>");
        return;
    }

    // Extraí registro e valor
    String regStr = input.substring(0, spaceIndex);
    String valueStr = input.substring(spaceIndex + 1);

    // Converte para inteiros
    int reg = regStr.toInt();
    int value = valueStr.toInt();

    // Envia comando Modbus
    if (writeData(0x1, reg, value)) {
        Serial.print("Comando enviado com sucesso para o registrador ");
        Serial.println(reg);
    } else {
        Serial.print("Erro ao enviar comando para o registrador ");
        Serial.println(reg);
    }
}
```

writeData function (Modbus Writing):

```
bool writeData(int addr, int reg, int value) {
    // Escreve no registrador de holding
    if (!ModbusRTUClient.holdingRegisterWrite(addr, reg, value)) {
```

```

        Serial.println("Erro ao enviar comando. Código de erro: " +
                        String(ModbusRTUClient.lastError()));
    return false;
}
return true;
}

```

Function readdata (Read - not implemented):

```

// Esta função não está implementada no código fornecido
// Seria necessária para leitura de dados do dispositivo
float readdata(int addr, int reg) {
    // Implementação exemplo:
    // return ModbusRTUClient.holdingRegisterRead(addr, reg);
    return 0.0;
}

```

6. System Operation

This code implements a Modbus RTU master with the following characteristics:

Modbus RTU Protocol:

- **Device Address:** 0x01 (default in the code)
- **Baud Rate:** 38400
- **Serial Configuration:** 8 data bits, no parity, 2 stop bits (SERIAL_8N2)
- **Register Type:** Holding Registers

Operation Flow:

1. **Initialization:** Configures RS485 communication with calculated delays
2. **Waiting for Commands:** Waits for input from the Serial Monitor
3. **Processing:** Interprets commands in the format "<register> <value>"
4. **Communication:** Sends Modbus commands to the slave device
5. **Feedback:** Reports success or failure of the operation

Example of Usage:

In the Serial Monitor, enter:

10 1

The above value will turn on the air conditioning.

In the Serial Monitor, enter:

15 0

The above value will turn off the air conditioning.